

# Microcontroladores: Laboratorio 2

1<sup>st</sup> Hector Pereira

Ingeniería en Mecatrónica

Universidad Tecnológica (UTEC)

Fray Bentos, Uruguay

hector.pereira@estudiantes.utec.edu.uy

2<sup>nd</sup> Isaac Martirena

Ingeniería en Mecatrónica

Universidad Tecnológica (UTEC)

Fray Bentos, Uruguay

isaac.martirena@estudiantes.utec.edu.uy

## Resumen—

## Keywords:

### I. INTRODUCCIÓN

I-A. Propósito del laboratorio

I-B. Descripción general de los módulos

I-C. Competencias desarrolladas

### II. MARCO TEÓRICO

II-A. Procesamiento de señales y sensores de color

Un sensor de luz o *Light Dependent Resistor* (LDR) es un dispositivo fotoresistivo cuya resistencia eléctrica disminuye cuando aumenta la intensidad luminosa incidente. Este comportamiento se debe a que la radiación electromagnética libera portadores de carga en el material semiconductor —generalmente sulfuro de cadmio— reduciendo su resistividad [1]. De este modo, el LDR convierte la variación de luz en una señal eléctrica analógica que puede ser interpretada por el convertidor analógico-digital (ADC) del microcontrolador.

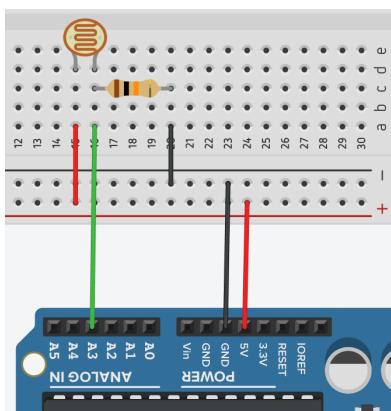


Figura 1. Esquema de conexión del sensor LDR en un divisor resistivo cuya salida se dirige al canal ADC del microcontrolador ATmega328P. Fuente: <https://paraarduino.com/sensores/fotorresistencias/valor-maximo-de-una-fotorresistencia/>.

En el caso del ATmega328P, el ADC posee una resolución de 10 bits, lo que permite representar las señales

analógicas dentro de un rango discreto de 0 a 1023, donde los valores extremos corresponden a la tensión mínima y máxima de referencia [2]. Estos niveles cuantificados pueden relacionarse con la intensidad relativa de los componentes del modelo aditivo de color **RGB** (*Red*, *Green*, *Blue*), el cual describe los colores como combinaciones de tres luces primarias. En dicho modelo, cada componente adopta un valor dentro de una escala de 0 a 255, rango que se utiliza en la representación digital del color en dispositivos emisores de luz, como pantallas o tiras de LEDs [3].

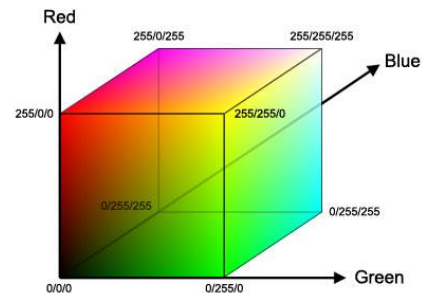


Figura 2. Representación tridimensional del modelo de color RGB, donde cada eje corresponde a una de las componentes primarias (*Red*, *Green*, *Blue*) y su combinación genera el espacio de colores aditivos. Fuente: [https://www.researchgate.net/figure/a-RGB-Color-Space-7-b-YCbCr-Color-Space-8\\_fig1\\_298734907](https://www.researchgate.net/figure/a-RGB-Color-Space-7-b-YCbCr-Color-Space-8_fig1_298734907).

Dado que la respuesta espectral del LDR no es lineal ni uniforme frente a distintas longitudes de onda, la interpretación cromática requiere un proceso de calibración que establezca correspondencias entre los valores eléctricos medidos y los tonos del espacio RGB. Este principio permite compensar las variaciones inherentes del sensor y asegurar una medición coherente en condiciones de iluminación variables.

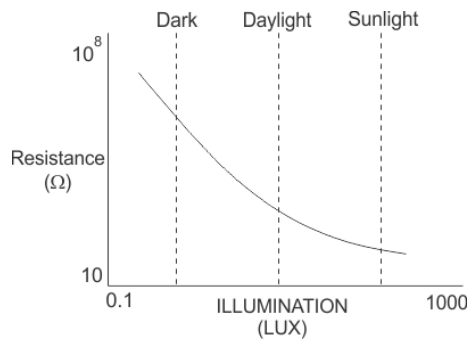


Figura 3. Curva típica de resistencia frente a la iluminación en un sensor LDR. Se observa el comportamiento no lineal del dispositivo, cuya resistencia disminuye de forma exponencial al aumentar la intensidad luminosa. Fuente: [https://www.researchgate.net/figure/Resistance-vs-illumination-curve-LDRs-are-light-dependent-devices-whose-resistance\\_fig3\\_318029856](https://www.researchgate.net/figure/Resistance-vs-illumination-curve-LDRs-are-light-dependent-devices-whose-resistance_fig3_318029856).

## II-B. Señales PWM y control de servomotores

El *Pulse Width Modulation* (PWM) o modulación por ancho de pulso es una técnica utilizada en sistemas embebidos para generar señales de control analógicas a partir de salidas digitales [3]. Consiste en emitir una señal periódica de frecuencia constante cuyo ciclo de trabajo (*duty cycle*) varía en función de la magnitud deseada. Este ciclo se define como la relación entre el tiempo en nivel alto ( $t_{on}$ ) y el periodo total ( $T$ ):

$$D = \frac{t_{on}}{T} \times 100 (\%)$$

En los servomotores de tipo *hobby*, el control de posición se basa en la variación del ancho del pulso dentro de un periodo fijo de aproximadamente 20 ms, correspondiente a una frecuencia de 50 Hz [4]. Pulsos de alrededor de 1 ms y 2 ms suelen representar los límites de desplazamiento angular del eje (por ejemplo, 0° y 180°). Así, la posición del servo se determina proporcionalmente al tiempo que la señal permanece en nivel alto.

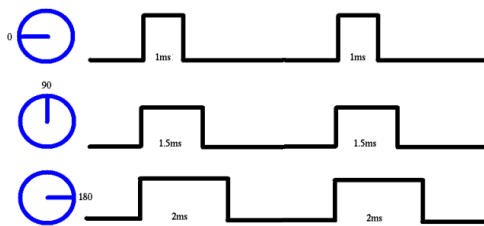


Figura 4. Señal PWM utilizada para el control de un servomotor de tipo *hobby*. La variación del ancho de pulso dentro de un periodo fijo de 20 ms determina la posición angular del eje, donde pulsos de aproximadamente 1 ms, 1.5 ms y 2 ms corresponden a 0°, 90° y 180°, respectivamente. Fuente: <https://electronics.stackexchange.com/questions/346603/driving-servo-motor-with-pwm-signal>.

En el microcontrolador ATmega328P, la generación de señales PWM se realiza mediante los temporizadores de hardware (*Timer/Counter*) [2]. Estos permiten obtener pulsos de duración precisa sin requerir intervención constante del

procesador, lo que garantiza estabilidad temporal, repetitividad y eficiencia computacional. Al delegar la modulación al hardware, el microcontrolador puede atender otras tareas concurrentes sin afectar la calidad de la señal de control.

## II-C. Reproducción de sonido digital

El sonido es una onda mecánica periódica cuya frecuencia determina la percepción del tono musical. En términos físicos, la frecuencia ( $f$ ), medida en hertzios (Hz), representa el número de oscilaciones por segundo, mientras que su inverso define el periodo ( $T = 1/f$ ) [1]. En los sistemas electrónicos, este principio se aplica mediante la generación de señales eléctricas alternantes que hacen vibrar un transductor, como un buzzer piezoeléctrico, produciendo así ondas sonoras audibles.

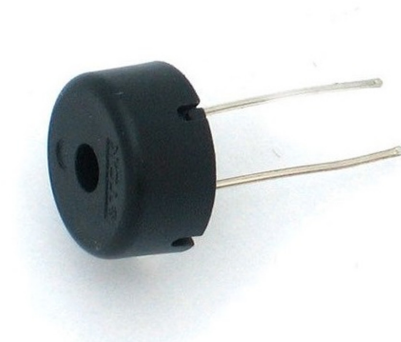


Figura 5. Buzzer piezoeléctrico modelo PS1240, utilizado como transductor acústico para la generación de sonido mediante señales cuadradas. Su funcionamiento se basa en la vibración mecánica del material piezoeléctrico al aplicarse una señal alternante. Fuente: [https://www.pishop.us/product/piezo-buzzer-ps1240/?srsltid=AfmBOooScrJUiyvIt6zUSB8B502aj-LKG3PUMQqgUtYcYVKXt\\_-yDs2](https://www.pishop.us/product/piezo-buzzer-ps1240/?srsltid=AfmBOooScrJUiyvIt6zUSB8B502aj-LKG3PUMQqgUtYcYVKXt_-yDs2).

En los microcontroladores, la síntesis de sonido se logra a través de la modulación temporal de señales digitales, comúnmente en forma de ondas cuadradas. La frecuencia de conmutación de estas ondas determina el tono percibido, mientras que su duración controla la extensión temporal de la nota [3]. De esta manera, es posible representar una escala musical a partir de un conjunto discreto de frecuencias asociadas a las notas estándar.

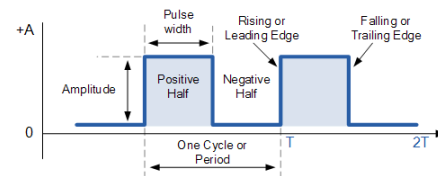


Figura 6. Onda cuadrada periódica empleada para la generación de tonos audibles en un buzzer piezoeléctrico. Se indican el periodo ( $T$ ) y la frecuencia ( $f = 1/T$ ), parámetros fundamentales que determinan el tono y la duración de la nota reproducida. Fuente: <https://www.allcoelec.com/blog/on-sinusoidal-waveforms-square,rectangular-and-pulsed-waveforms.html>.

En el ámbito de la música digital, el formato **MIDI** (*Musical Instrument Digital Interface*) constituye un estándar ampliamente utilizado para describir eventos musicales mediante códigos numéricos [5]. Cada evento MIDI contiene información como la nota a ejecutar, su duración, intensidad y canal asignado. Aunque no transmite sonido directamente, este formato permite definir secuencias musicales que pueden ser interpretadas por distintos dispositivos electrónicos, incluidos los microcontroladores, mediante la generación de las frecuencias correspondientes.

La reproducción de melodías digitales se basa, por tanto, en la secuenciación de notas definidas por su frecuencia y duración. Este enfoque, derivado de los principios del formato MIDI, permite sintetizar melodías simples mediante la temporización de señales cuadradas, demostrando la capacidad de los sistemas embebidos para generar sonido a partir de parámetros discretos de control temporal y frecuencia.

#### II-D. Interfaz de usuario e interacción hombre-máquina

En los sistemas embebidos, la **interfaz de usuario** (*User Interface*, UI) se define como el conjunto de componentes físicos y lógicos que facilitan la comunicación entre el usuario y el dispositivo. Su función principal es transformar las acciones del operador en respuestas visuales o auditivas que reflejen el estado interno del sistema, permitiendo un control intuitivo y seguro del mismo [6].

Este intercambio bidireccional se logra a través de distintos medios de entrada y salida. Los periféricos de entrada —como teclados, pulsadores o sensores— registran las acciones del usuario, mientras que los de salida —como pantallas, indicadores luminosos o buzzers— proporcionan *feedback* sobre el resultado de dichas acciones. En conjunto, estos elementos conforman un canal de comunicación que mejora la interacción y la comprensión del funcionamiento del sistema [3].

Para organizar la lógica de funcionamiento, los sistemas embebidos suelen emplear una **máquina de estados finitos** (FSM, por sus siglas en inglés). Este modelo conceptual divide la operación del programa en un conjunto finito de estados definidos, entre los cuales se producen transiciones en respuesta a eventos determinados [7]. Dicho enfoque favorece la modularidad, la claridad del código y la predictibilidad del comportamiento, cualidades esenciales en el diseño de interfaces de usuario para entornos con recursos limitados.

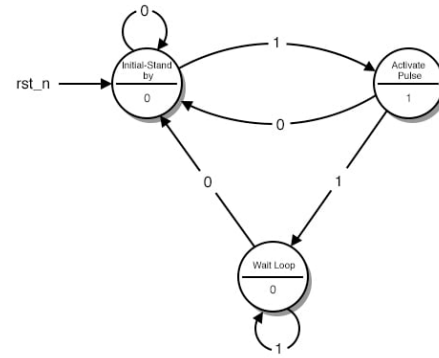


Figura 7. Ejemplo de diagrama de una máquina de estados finitos (FSM), donde cada nodo representa un estado lógico del sistema y las transiciones se activan por eventos o condiciones externas. Este enfoque permite modelar el comportamiento secuencial y predecible de una interfaz de usuario embebida. Fuente: <https://www.allaboutcircuits.com/textbook/digital/chpt-11/finite-state-machines/>.

#### II-E. Almacenamiento no volátil y EEPROM

La **EEPROM** (*Electrically Erasable Programmable Read-Only Memory*) es un tipo de memoria no volátil que permite conservar la información almacenada incluso después de interrumpir la alimentación del sistema [3]. A diferencia de la memoria RAM, que se borra al apagar el dispositivo, la EEPROM mantiene los datos de forma permanente hasta que son modificados explícitamente por el programa, lo que la hace ideal para guardar configuraciones o valores críticos de usuario.

En el microcontrolador ATmega328P, este tipo de memoria tiene una capacidad de 1 kB organizada en celdas de 8 bits, cada una de las cuales puede ser escrita y borrada eléctricamente un número limitado de veces —del orden de  $10^5$  ciclos— según las especificaciones del fabricante [2]. Debido a esta limitación, es recomendable minimizar las operaciones de escritura para evitar el desgaste prematuro de las celdas y asegurar la longevidad del dispositivo.

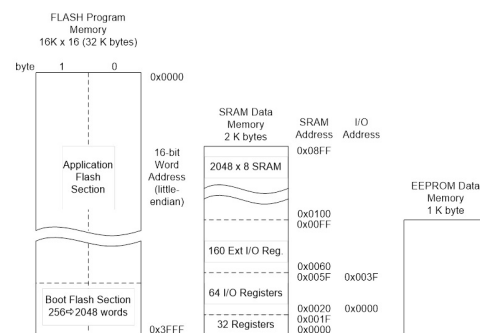


Figura 8. Estructura jerárquica de memoria del microcontrolador ATmega328P, que incluye los espacios de memoria *Flash*, *SRAM* y *EEPROM*. Cada uno cumple una función específica: la memoria *Flash* almacena el programa, la *SRAM* gestiona las variables temporales en ejecución y la *EEPROM* conserva datos no volátiles. Fuente: <https://www.arxterra.com/3-addressing-modes/>.

La EEPROM se utiliza comúnmente en sistemas embebidos para conservar información que debe persistir entre en-

cendidos, como parámetros de calibración, configuraciones del sistema o datos de usuario. Su manejo puede realizarse mediante instrucciones específicas del microcontrolador o por medio de librerías de más alto nivel que abstraen las operaciones de lectura y escritura, garantizando un acceso controlado y confiable a la memoria no volátil [8].

## II-F. Planificación temporal y multitarea cooperativa

En los sistemas embebidos, la ejecución de múltiples procesos requiere una estrategia de **planificación temporal** que permita coordinar distintas tareas sin provocar bloqueos ni retardos innecesarios [3]. Una de las técnicas más empleadas en microcontroladores de propósito general es la **multitarea cooperativa**, en la cual las tareas se ejecutan de manera secuencial dentro del ciclo principal (*main loop*) y ceden voluntariamente el control al resto una vez completada su función [7].

A diferencia de los sistemas con planificación basada en interrupciones o sistemas operativos en tiempo real (RTOS), la multitarea cooperativa no requiere mecanismos de cambio de contexto ni gestión de múltiples pilas, lo que la hace especialmente adecuada para dispositivos con recursos limitados. En este modelo, cada proceso es responsable de ejecutarse rápidamente y permitir la continuidad del flujo general del programa, garantizando un funcionamiento fluido y determinista [6].

Esta técnica permite estructurar el programa en módulos independientes que se ejecutan de forma periódica o condicional, favoreciendo un diseño más claro y eficiente. Su simplicidad y bajo consumo de recursos la convierten en una herramienta fundamental en el desarrollo de sistemas embebidos donde la sincronización temporal es crítica y la capacidad de procesamiento es reducida.

## III. METODOLOGÍA

### III-A. Materiales y Herramientas

- Microcontrolador ATmega328P (Arduino Uno)
- Sensor LDR y LED RGB
- Servomotor SG90
- Teclado matricial 4x4
- Pantalla LCD 16x2 con interfaz I2C
- Buzzers piezoeléctricos (activo y pasivo)
- Tira LED WS2812
- Resistencias, cables, protoboard
- Software: Microchip Studio, Proteus / PicSimLab, Python (para generación de secuencias del plotter)

### III-B. Procedimiento general

Se dividió el sistema en cuatro módulos independientes (Plotter, Colores, Piano y Cerradura) y se abarcó cada consigna de manera individual.

#### III-B1. Plotter:

#### III-B2. Colores:

**Idea general:** — El circuito se implementó mediante un divisor resistivo conectado a una de las entradas analógicas del microcontrolador. El color identificado se replica visualmente en la tira de LED WS2812, y el servomotor se posiciona en el ángulo correspondiente (predefinido) al color detectado dentro de la hoja de referencia, integrando así un sistema de selección e identificación de color completamente automatizado.

**Descomposición de colores:** — El proceso de reconocimiento de colores se fundamenta en la descomposición de cada color en sus componentes primarias dentro del modelo RGB (*Red, Green, Blue*). Un color puede representarse como la combinación ponderada de estas tres intensidades, lo que permite su descripción en un espacio tridimensional.

Sin embargo, el sensor fotoresistivo (LDR, *Light Dependent Resistor*) utilizado en el sistema no distingue de forma individual las componentes cromáticas del espectro visible; únicamente mide la intensidad total de luz reflejada sobre su superficie. Si la iluminación se realiza con luz blanca, el sensor solo entrega una lectura proporcional a la cantidad total de luz reflejada, sin discriminar su composición espectral. Este fenómeno equivale a una percepción en escala de grises, dificultando la identificación precisa de colores.

Para resolver esta limitación, se empleó un diodo emisor de luz RGB como fuente de iluminación controlada. Al iluminar secuencialmente la superficie con luz roja, verde y azul, el sistema obtiene tres mediciones independientes mediante el LDR. Dichos valores, convertidos por el módulo ADC (*Analog-to-Digital Converter*) del microcontrolador ATmega328P, representan las coordenadas ( $R, G, B$ ) de un punto dentro de un espacio de color tridimensional (véase anexo A2b).

**Calibración y reconocimiento:** — Dado que tanto la respuesta espectral del LDR como la emisión de los LED presentan variaciones no lineales, los valores obtenidos no son directamente proporcionales a las componentes RGB teóricas. No obstante, se implementa un proceso de calibración inicial para contrarrestar estos efectos, donde se miden manualmente los colores de referencia (rojo, verde, azul claro, violeta, morado, amarillo y blanco) y se almacenan sus valores de respuesta RGB de conversión analógica-digital.

Cada color de referencia calibrado se modela como un vector fijo en dicho espacio. Para identificar un color, se calcula la distancia euclidiana entre el vector de medición y cada uno de los vectores de referencia almacenados:

$$D = \sqrt{(R_m - R_i)^2 + (G_m - G_i)^2 + (B_m - B_i)^2} \quad (1)$$

donde  $(R_m, G_m, B_m)$  representan los valores medidos por el sensor y  $(R_i, G_i, B_i)$  corresponden a los valores calibrados de cada color de la hoja de referencia. El color identificado será aquel cuya distancia  $D$  sea mínima (véase anexo A2f).

**Servomotor y tira LED:** — El servomotor se controla mediante el *Timer1* configurado en modo *Fast PWM* a 50 Hz, generando pulsos de entre 1 y 2 ms para posicionar el eje según el color identificado. Cada ángulo se calcula a partir del valor del registro *OCR1A*, que define el tiempo en alto del ciclo PWM (véase anexo A2e).

La visualización del color se realiza con una tira de LED WS2812, donde cada diodo recibe datos digitales codificados en tres bytes (RGB). Para cumplir con los tiempos estrictos del protocolo, se emplearon instrucciones *asm volatile* que garantizan una sincronización precisa en la transmisión de pulsos. Así, al identificar un color, el sistema replica su tonalidad en la tira LED y mueve el servomotor al ángulo correspondiente, integrando una respuesta visual y mecánica simultánea (véase anexo A2d).

**USART:** — A través de USART se comunican los valores del sensor y el color detectado hacia una interfaz serial. Se utiliza un buffer circular para transmisión continua sin bloqueo, y la función auxiliar *UTOA* convierte los datos numéricos del ADC en texto ASCII antes de su envío. De esta forma, el sistema comunica en tiempo real los valores (*R, G, B*) medidos, el color identificado y el ángulo del servomotor, facilitando el monitoreo y la calibración del sistema (véase anexo A2g).

### III-B3. Piano:

**Notas musicales:** — El sonido es una onda mecánica que se propaga a través de un medio elástico, producto de variaciones periódicas de presión. Estas ondas pueden clasificarse según su forma en senoidales, cuadradas, triangulares o diente de sierra, dependiendo del patrón temporal de oscilación. En los sistemas digitales, las señales más sencillas de generar son las ondas cuadradas, donde el voltaje alterna entre dos niveles definidos, representando los estados lógicos alto y bajo del microcontrolador.

Para producir sonido de manera electrónica, se emplean dispositivos piezoeléctricos denominados *buzzers*. Estos transductores convierten la energía eléctrica en vibraciones mecánicas audibles. Cuando el microcontrolador aplica una señal cuadrada a la entrada del buzzer, el material piezoeléctrico se deforma y contrae periódicamente, generando un sonido cuya frecuencia está directamente relacionada con la frecuencia de la señal de entrada.

En el microcontrolador ATmega328P, esta señal se genera mediante el uso de los temporizadores internos (*timers*), configurados para producir una onda cuadrada en un pin de salida. Al modificar la frecuencia de conmutación, es posible variar el tono percibido, ya que la frecuencia del sonido determina su altura musical. De esta manera, cada nota puede asociarse a una frecuencia específica según la escala. Por ejemplo, la nota *La4* corresponde a una frecuencia de 440 Hz, mientras que *Do4* equivale aproximadamente a 262 Hz (véase anexo A3b).

**Generación de frecuencias:** — Para reproducir una nota musical, el microcontrolador debe generar una señal cuadrada con una frecuencia específica, determinada por el valor de comparación del temporizador. En el ATmega328P, dicha frecuencia depende de la relación entre la frecuencia del reloj del sistema (*F\_CPU*), el valor de comparación *OCRx*, y el prescaler aplicado al temporizador. Este último actúa como un divisor de frecuencia, reduciendo la velocidad a la que el contador incrementa.

Elegir el prescaler adecuado resulta fundamental para asegurar que el valor de *OCRx* se mantenga dentro del rango permitido por el temporizador (por ejemplo, 8 bits en el *Timer0*). Si la frecuencia deseada es alta, se requiere un prescaler pequeño para mantener precisión; si es baja, un prescaler grande evita desbordamientos y permite generar tonos audibles.

En la práctica, el programa selecciona dinámicamente el prescaler que produce un valor de *OCR0A* válido para la nota solicitada (véase anexo A3c). De esta forma, es posible generar un amplio rango de frecuencias musicales con un solo temporizador, manteniendo una relación estable entre tono y duración.

**Reproducir música:** — Una nota musical puede representarse computacionalmente mediante tres parámetros fundamentales: la frecuencia de oscilación, la duración temporal durante la cual se mantiene activa y la duración inactiva o en silencio. Cuando el buzzer emite una secuencia ordenada de notas, cada una con su frecuencia, tiempo de activación, y silencio, se obtiene una melodía. En términos programáticos, una melodía se define como un conjunto de estructuras de datos que contienen pares (*frecuencia*, *tiempo\_on*, *tiempo\_off*), los cuales son procesados secuencialmente para generar la música deseada.

Para crear pistas de canciones se utilizó la herramienta *MIDI to Arduino Converter* [9], la cual permite convertir pistas MIDI al formato anteriormente mencionado, y almacenarlas dentro de la memoria del microcontrolador.

La reproducción simultánea de varios instrumentos en un entorno digital requiere el manejo de múltiples secuencias de notas en paralelo. Para lograrlo sin recurrir al uso de ciclos de espera activos (*polling*), se utilizan interrupciones asociadas a los temporizadores. De este modo, el microcontrolador puede controlar la duración y el inicio de cada nota de forma independiente y precisa, optimizando el uso del procesador y permitiendo la ejecución de tareas concurrentes, como la lectura de entradas o la comunicación serial, mientras la música continúa sonando de manera autónoma (véase anexo A3d).<sup>9</sup>

**Modos de funcionamiento y USART:** — Para implementar el modo piano se decidió utilizar 8 pulsadores correspondientes a las 8 principales notas musicales. Cada uno de los pulsadores fue conectado a un pin del puerto D

del microcontrolador, configurando los pines como entradas y pull-ups internas. Utilizando interrupciones externas por cambio de pin (PCINT) y lógica programática para debouncing, se implementó la funcionalidad de que mientras el pulsador está presionado, la nota sigue sonando.

Finalmente, a este sistema se integra funcionalidad USART para mostrar un menú al usuario por consola y permitir comandar el funcionamiento del sistema de manera sencilla. El usuario es presentado con 3 opciones: [1]: Reproducir canción 1, [2]: Reproducir canción 2, [P]: Cambiar a modo piano. Mediante variables de control se deshabilita la funcionalidad de piano durante la reproducción de canciones, y vice versa, al cambiar al modo piano se detiene y reinician las pistas que se estaban reproduciendo (véase anexo A3e).

En síntesis, el sistema desarrollado utiliza un buzzer piezoeléctrico controlado por los temporizadores del ATmega328P para reproducir notas musicales definidas por su frecuencia y duración. A través de la programación de secuencias almacenadas en memoria, es posible interpretar melodías completas y gestionar la reproducción de distintas canciones sin intervención del usuario durante la ejecución. Utilizando USART se puede comandar al programa para reproducir diferentes pistas guardadas o cambiar a modo piano donde 8 pulsadores reproducen las 8 notas principales.

### III-B4. Cerradura:

**Idea general:** — El sistema de cerradura electrónica desarrollado tiene como objetivo controlar el acceso mediante una contraseña numérica almacenada en memoria no volátil. Inicialmente, el dispositivo se encuentra en estado de *candado cerrado*. Cuando el usuario ingresa la contraseña correcta, el sistema cambia al estado de *candado abierto*, permitiendo el acceso. En caso de introducir una contraseña incorrecta tres veces consecutivas, se activa una alarma acústica a través de un buzzer, indicando un intento de acceso no autorizado.

El sistema también permite modificar la contraseña almacenada. Para ello, el usuario debe ingresar primero la contraseña actual y, tras su validación, definir una nueva contraseña de entre cuatro y seis dígitos. Esta nueva clave se almacena permanentemente en la memoria EEPROM del microcontrolador, garantizando su persistencia incluso después de un apagado o reinicio del sistema.

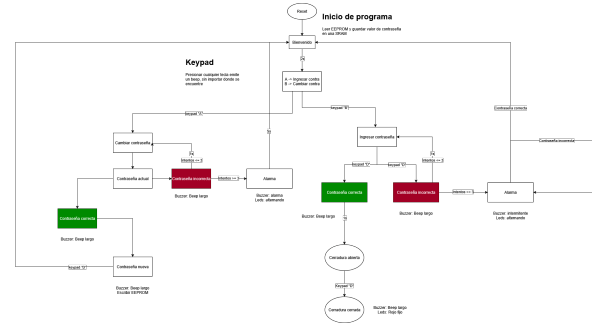


Figura 9. Diagrama de flujo de programa de cerradura. Fuente: elaboración propia

**Interfaz de usuario:** — La cerradura dispone de una interfaz de usuario compuesta por una pantalla LCD de 16x2, un teclado matricial 4x4 y tres indicadores luminosos (LED verde, LED rojo y alarma sonora). En este contexto, la interfaz constituye el medio de comunicación entre el usuario y el sistema, mostrando mensajes informativos y recibiendo acciones por medio del teclado. Cada interacción del usuario genera una respuesta visual o acústica distinta, representando así un flujo de diálogo entre ambos.

El sistema se diseñó siguiendo una lógica de *máquina de estados finitos*, en la que cada modo de operación (menú principal, ingreso de contraseña, cambio de clave, acceso autorizado, alarma, etc.) representa un estado. Las transiciones entre estados se producen en función de las acciones del usuario y las condiciones del sistema. Este enfoque facilita la gestión de comportamientos complejos, simplifica el control del flujo de ejecución y mejora la legibilidad del código (véase anexo A4b).

**Teclado matricial:** — El ingreso de datos se realiza mediante un teclado matricial 4x4 que combina filas y columnas, optimizando el uso de pines del microcontrolador. Las filas se configuran como salidas y las columnas como entradas con resistencias de *pull-up* activadas. El proceso de lectura consiste en activar una fila a nivel bajo (0) mientras las demás permanecen en alto (1); si alguna tecla de esa fila se encuentra presionada, la columna correspondiente cambia a nivel bajo, permitiendo identificar la intersección entre fila y columna (véase anexo A4c).

Cada tecla se asocia a un carácter según su posición dentro de la matriz, lo que permite retornar el valor numérico o simbólico correspondiente. Para evitar falsas detecciones debido al rebote mecánico de los contactos, se aplica una rutina de *debouncing* por software basada en pequeños retardos temporizados.

**Planificador de tareas:** — Durante el funcionamiento, el sistema debe ejecutar múltiples tareas de forma paralela: reproducción de sonidos, manejo de alarmas, parpadeo de LEDs, lectura del teclado y actualización de la pantalla LCD. Sin embargo, el ATmega328P dispone de un número



| Color      | R   | G   | B   |
|------------|-----|-----|-----|
| Morado     | 216 | 157 | 274 |
| Morajo     | 206 | 149 | 262 |
| Amarillo   | 196 | 99  | 105 |
| Verde      | 272 | 192 | 152 |
| Azul Claro | 151 | 153 | 122 |
| Violeta    | 253 | 275 | 338 |
| Blanco     | 110 | 93  | 91  |

### Reconocimiento decolores y manejo de la tira LED:

Además, se observó que la cantidad de luz ambiental influía directamente en la precisión de la lectura del sensor. La misma tira de LEDs WS2812 utilizada para la representación del color introducía una leve interferencia óptica sobre la LDR cuando se encontraba demasiado cerca, provocando pequeñas desviaciones en las mediciones. Para mitigar este efecto, se implementó un apantallamiento parcial que bloqueó la incidencia directa de la luz de la tira sobre el sensor, manteniendo al mismo tiempo una iluminación uniforme sobre la superficie de calibración. Con estas modificaciones, el sistema logró una detección estable y una correspondencia confiable entre los valores ADC medidos y los colores calibrados.

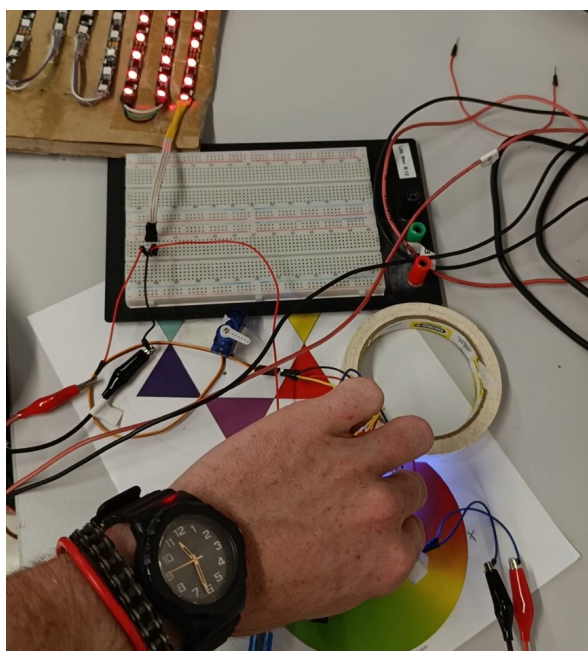


Figura 11. Identificación de color efectiva. Fuente: elaboración propia

### Comunicación USART:

Finalmente, durante la implementación de la comunicación serial mediante USART se detectó una ralentización perceptible en el ciclo principal del programa. Esto se debía al tiempo requerido para transmitir cada paquete de datos hacia la terminal, especialmente cuando se imprimían valores en cada iteración del bucle. Para evitar sobrecargas en el búfer de transmisión (*overflow*) y asegurar una comunicación confiable, fue necesario introducir retardos (*delay*) calibrados entre envíos sucesivos, limitando así la frecuencia de actualización sin comprometer la precisión del monitoreo. La Figura 12 muestra la salida obtenida en la terminal serial, donde se visualizan el valor analógico medido, el color identificado, el valor de referencia y la desviación asociada.

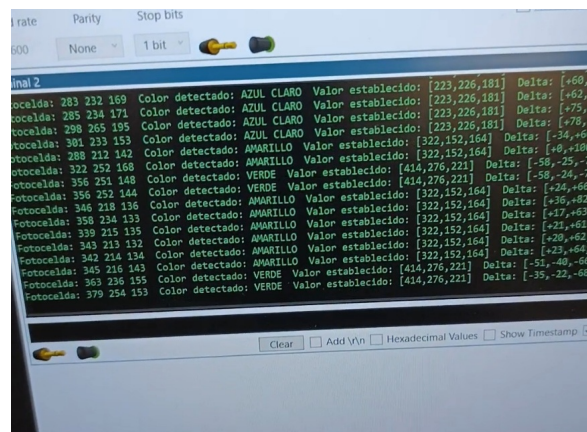


Figura 12. Identificación de color efectiva mediante comunicación serial a través de la terminal. Se observa la lectura del valor ADC, el color detectado, el valor establecido y la desviación del mismo. Fuente: elaboración propia.

### IV-A3. Piano: —

#### Simulación: —

La simulación en **Proteus** permitió verificar el funcionamiento general del sistema de piano electrónico antes del montaje físico. Se observó que al accionar los pulsadores se generaban las notas esperadas y que las interrupciones por cambio de pin respondían de forma estable, reproduciendo correctamente los tonos tanto en modo manual como en modo automático. El uso del microcontrolador **ATmega328P** de forma independiente, en lugar del módulo **Arduino Uno**, corrigió los problemas iniciales con las interrupciones **PCINT**, logrando una respuesta fluida en la reproducción de las melodías.

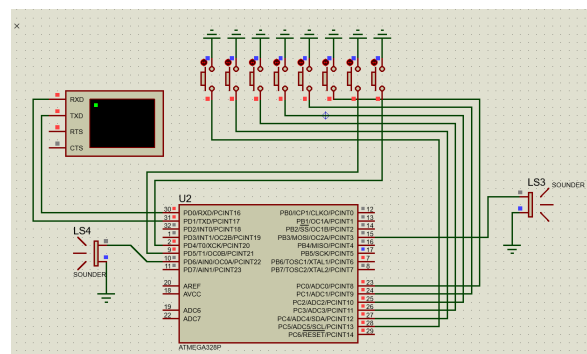


Figura 13. Simulación en Proteus del sistema de piano electrónico. Se observa la activación de los pulsadores, la generación de tonos en el buzzer y la respuesta del microcontrolador ATmega328P configurado con interrupciones **PCINT**. Fuente: elaboración propia.

#### Generación de frecuencias: —

Durante las pruebas físicas, el sistema generó correctamente las frecuencias correspondientes a cada nota musical. Las señales cuadradas producidas fueron estables y sin distorsiones audibles, manteniendo una relación precisa entre la frecuencia programada y el tono percibido. El buzzer respondió con buena fidelidad en todo el rango de notas implementado, sin interferencias ni retardos perceptibles al presionar las teclas.



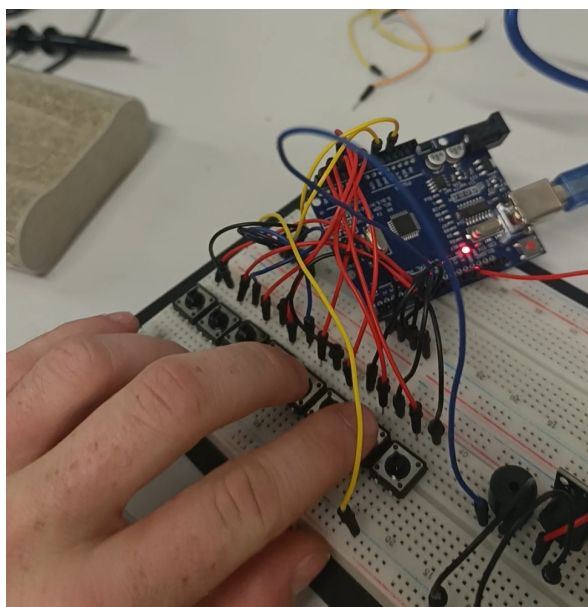


Figura 14. Montaje del circuito correspondiente al módulo *Piano*, compuesto por el microcontrolador ATmega328P, el buzzer piezoeléctrico, los pulsadores de entrada y los componentes de conexión. Fuente: elaboración propia.

#### *Reproducción de música: —*

El sistema reprodujo correctamente las melodías preprogramadas, manteniendo el ritmo y la secuencia esperada de las notas. Se detectaron leves desfasajes entre las pistas durante la ejecución, lo cuales se volvían más aparentes conforme avanzaba la reproducción. Para melodías con una sola pista no se detectaron desviaciones con las originales, lo que confirmó la estabilidad temporal del módulo. Las canciones fueron reproducidas con continuidad y volumen uniforme, evidenciando un control adecuado sobre la temporización y la generación de frecuencias.

#### *Modos de funcionamiento y comunicación USART:*

Durante la verificación de los modos de operación, se comprobó que el sistema respondía correctamente tanto en el modo **manual** (piano) como en el modo **automático** (canciones). La interfaz serial **USART** permitió alternar entre los modos de forma confiable, así como iniciar y detener la reproducción desde la terminal. La comunicación se mantuvo estable a 9600 bps, sin pérdida de datos ni interferencia con la reproducción del sonido. El menú mostrado en la terminal facilitó el control y la supervisión del sistema durante las pruebas.

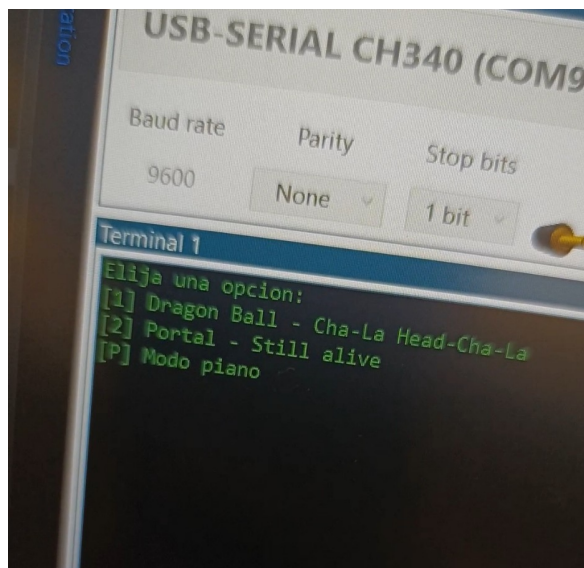


Figura 15. Interfaz de comunicación serial (*USART*) mostrando el menú de control del módulo *Piano*. Desde la terminal, el usuario puede seleccionar entre el modo manual por pulsadores o la reproducción automática de melodías predefinidas. Fuente: elaboración propia.

#### *IV-A4. Cerradura: —*

##### *Simulación: —*

La simulación del sistema se realizó inicialmente en **Proteus**, aunque se presentaron dificultades con la visualización en la pantalla LCD con interfaz I2C, la cual no respondió de forma adecuada a las señales del microcontrolador. Posteriormente, el circuito fue probado en **PicSimLab**, donde el sistema funcionó correctamente y permitió validar el funcionamiento del teclado matricial, la pantalla LCD, los LEDs indicadores y el buzzer. Durante las pruebas, el ingreso y verificación de contraseñas se comportaron de manera estable, y se observó la transición entre estados de bloqueo, desbloqueo y alarma. El almacenamiento en memoria EEPROM también pudo verificarse, conservando los datos tras los reinicios del sistema.

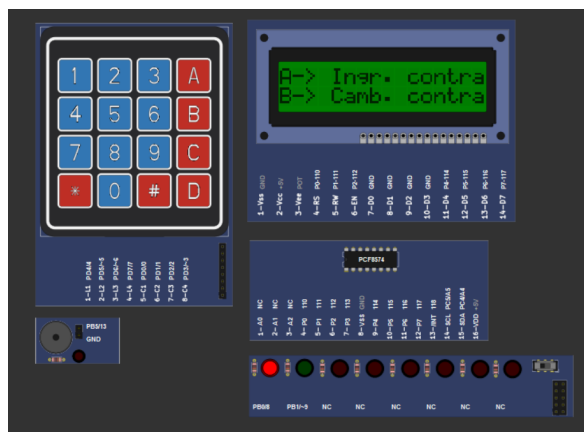


Figura 16. Simulación en PicSimLab del sistema de cerradura electrónica. Se observan los periféricos principales: teclado matricial, pantalla LCD, LEDs indicadores y buzzer, así como la correcta ejecución de las rutinas de interfaz y control de estado. Fuente: elaboración propia.

#### **Teclado matricial: —**

Durante la verificación del teclado matricial 4x4, se comprobó una lectura estable y sin errores de rebote, incluso con pulsaciones rápidas o repetitivas. El ingreso de la contraseña fue detectado con precisión, y el sistema respondió inmediatamente al comparar los dígitos con la clave almacenada. El comportamiento fue consistente tanto en simulación como en la prueba física, sin registros de lecturas duplicadas o fallas de detección.

#### **Interfaz de usuario: —**

En las pruebas de la interfaz LCD se confirmó que los mensajes se mostraban correctamente en cada etapa del proceso: inicio del sistema, solicitud de contraseña, acceso concedido o denegado y aviso de alarma. Tras tres intentos fallidos consecutivos, se activó la alarma sonora y el LED rojo de manera intermitente, mientras que el LED verde indicó exitosamente la apertura del sistema cuando la contraseña era correcta. La interacción visual y auditiva resultó clara y permitió identificar el estado del sistema en todo momento.

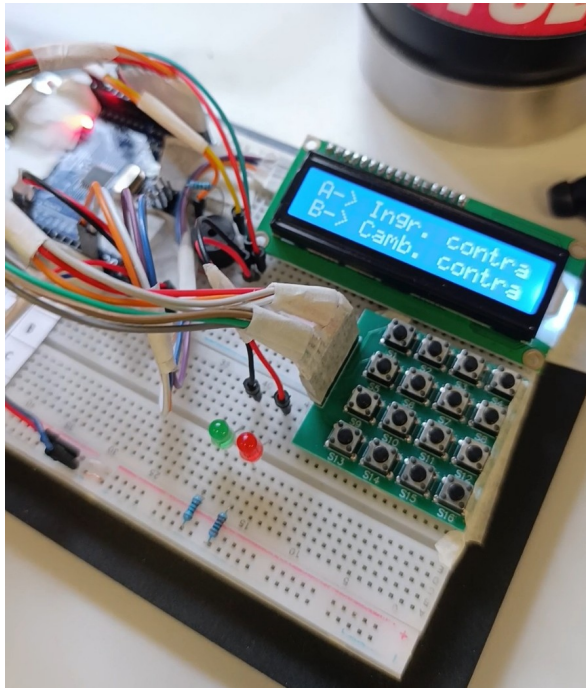


Figura 17. Montaje del circuito correspondiente al sistema de cerradura electrónica. Se observa la conexión del teclado matricial 4x4, la pantalla LCD 16x2 con interfaz I2C, los indicadores luminosos (LED rojo y verde), el buzzer de alarma y el microcontrolador ATmega328P. Fuente: elaboración propia.

#### **Planificador de tareas: —**

Durante las pruebas se verificó que el planificador de tareas permitió ejecutar de forma fluida las distintas funciones del sistema sin bloqueos perceptibles. El escaneo del teclado, la actualización del LCD, la supervisión de la alarma y el control de los LEDs se realizaron de manera sincronizada y estable, manteniendo una respuesta inmediata a las acciones del usuario. No se registraron conflictos temporales ni retrasos en la ejecución de las tareas programadas.

#### **Almacenamiento en EEPROM: —**

El sistema demostró un funcionamiento confiable del almacenamiento no volátil. La contraseña permaneció guardada tras múltiples reinicios, confirmando la correcta escritura y lectura en la memoria **EEPROM**. Durante las pruebas, se modificó la contraseña en varias ocasiones y los nuevos valores se conservaron correctamente, mostrando la robustez del procedimiento de guardado. En conjunto, el módulo cumplió con los objetivos previstos, garantizando una gestión segura y persistente de la información del usuario.

### **V. CONCLUSIONES**

#### **V-A. Colores**

El desarrollo del sistema de detección de colores permitió comprender en profundidad el funcionamiento del conversor analógico-digital (ADC) del microcontrolador y su aplicación en la lectura de un sensor LDR. Además, se logró implementar con éxito la comunicación con la tira de LEDs WS2812 y el control de un servomotor mediante señales PWM, integrando varios periféricos de manera coordinada.

Entre las posibles mejoras, se destaca la necesidad de optimizar la comunicación por USART, la cual limitó la velocidad de respuesta del sistema. También sería conveniente incorporar un modo de calibración que permita al usuario registrar valores de referencia personalizados para cada color, adaptando el sistema a diferentes condiciones de iluminación. Finalmente, el uso de un servomotor de 360° posibilitaría cubrir un rango mayor de colores en la rueda.

#### **V-B. Piano**

Este módulo permitió afianzar el manejo de interrupciones, temporizadores y la generación de señales de distinta frecuencia mediante la modulación por ancho de pulso. A partir de ello, se consiguió la reproducción tanto de notas individuales como de melodías almacenadas en memoria.

Como mejoras futuras, se propone ampliar el repertorio de canciones predefinidas y aumentar la cantidad de notas disponibles en el teclado, extendiendo el rango tonal del instrumento. Asimismo, podría trabajarse en la sincronización de las pistas para reducir el leve desfase que aparece durante la reproducción prolongada.

#### **V-C. Cerradura**

El desarrollo de la cerradura electrónica permitió integrar múltiples conceptos del curso: el uso de la memoria **EEPROM** para el almacenamiento persistente de contraseñas, la lectura de teclados matriciales con técnicas de *debouncing*, la creación de máquinas de estados y la gestión de tareas temporizadas mediante un solo temporizador.

Como mejoras, se plantea incorporar un botón que permita visualizar temporalmente la contraseña antes de confirmar, mejorando la experiencia del usuario sin comprometer la seguridad. Además, la inclusión de un solenoide o mecanismo físico real de bloqueo daría una respuesta tangible al

proceso de autenticación, completando el sistema de manera práctica.

## REFERENCIAS

- [1] A. S. Sedra and K. C. Smith, *Microelectronic Circuits*, 8th ed. Oxford University Press, 2021, conceptos de fotoconductividad y respuesta espectral en materiales semiconductores.
- [2] Microchip Technology Inc. Atmega328p datasheet. [Online]. Available: [https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P\\_Datasheet.pdf](https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf)
- [3] M. A. Mazidi, S. Naimi, and S. Naimi, *The AVR Microcontroller and Embedded Systems: Using Assembly and C*. Pearson Education, 2014, referencia clásica sobre arquitectura AVR, ADC, PWM y EEPROM.
- [4] J. Angeles, *Fundamentals of Robotic Mechanical Systems: Theory, Methods, and Algorithms*, 5th ed. Springer, 2018, referencias sobre control por PWM y cinemática de actuadores.
- [5] International MIDI Association, *MIDI 1.0 Detailed Specification*, IMA, 1996, documento técnico que define la codificación de eventos musicales en el formato MIDI.
- [6] E. Williams, *Make: AVR Programming: Learning to Write Software for Hardware*. O'Reilly Media, 2014, guía práctica de programación en C para microcontroladores AVR.
- [7] J. J. Labrosse, *Embedded Systems Building Blocks: Complete and Ready-to-Use Modules in C*. CRC Press, 2013, base teórica sobre máquinas de estados finitos y multitarea cooperativa en sistemas embebidos.
- [8] AVR Libc Development Team, *AVR Libc User Manual, Version 2.2.0*, Savannah GNU Project, 2024, documentación oficial de la biblioteca estándar de C para AVR. [Online]. Available: <https://www.nongnu.org/avr-libc/user-manual/index.html>
- [9] ShivamJoker, “Midi to arduino converter,” <https://arduinomidi.netlify.app/>, 2024, herramienta en línea para convertir archivos MIDI a código Arduino.
- [10] circuito.io. (2018) Arduino uno pinout diagram. [Online]. Available: <https://www.circuito.io/blog/arduino-uno-pinout/>
- [11] ARXterra. Addressing modes — 8-bit avr instruction set. [Online]. Available: <https://www.arxterra.com/3-addressing-modes/>
- [12] Software Particles. (2024, Apr.) Learn how an 8x8 led display works and how to control it using an arduino. Figura: 8x8 LED matrix pin mapping (imagen del artículo). [Online]. Available: <https://softwareparticles.com/learn-how-an-8x8-led-display-works-and-how-to-control-it-using-an-arduino/>
- [13] Carpeta del laboratorio (google drive). Carpeta compartida del laboratorio. [Online]. Available: <https://drive.google.com/drive/u/0/folders/1fP0aILozXeapRgDPDNWT1TRhAYr1PPPT>
- [14] Microchip Community (AVR Freaks). Avr freaks — comunidad de desarrolladores avr. Foros técnicos y soluciones prácticas sobre AVR. [Online]. Available: <https://www.avrfreaks.net/>
- [15] J. Ganssle. A guide to debouncing. Referencia clásica para anti-rebote en pulsadores/encoders. [Online]. Available: <https://www.ganssle.com/debouncing.htm>
- [16] G. Schmidt. (2021, Sep.) Beginners introduction to the assembly language of atmel-avr-microprocessors. Tutorial de avr-asm-tutorial.net (revisión septiembre 2021). [Online]. Available: <https://kitsandparts.com/tutorials/assemblers/BeginnersAVRasm.pdf>
- [17] T. Redelberger. (2019, Apr.) Avr assembler for complex projects. Versión 0.4 (2019-04-06). [Online]. Available: [https://web222.webclient5.de/doc/swdev/avrasm/advavrasm2/AdvancedUseAVRASM2\\_en\\_20190406.pdf](https://web222.webclient5.de/doc/swdev/avrasm/advavrasm2/AdvancedUseAVRASM2_en_20190406.pdf)
- [18] Laboratorio de Microcontroladores, UTEC, “Repositorio de laboratorio de microcontroladores (tec.micro),” <https://github.com/MateoLecuna/Tec.Micro>, 2025, accedido el 26 de septiembre de 2025.

## APÉNDICE

### A. Fragmentos de código

A1. *Plotter*: —

A2. *Colores*: —

A2a. *Librerías utilizadas*: —

```
#define F_CPU 16000000UL

#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
#include <string.h>
```

Listing 1. Librerías utilizadas

A2b. *Lectura de colores*: —

```
uint8_t led_state = 0;
uint16_t adc_sample[] = {0,0,0};

// Leer adc
uint16_t adc_read(uint8_t channel) {
    ADMUX = (ADMUX & 0xF0) | (channel & 0x0F);
    ADCSRA |= (1 << ADSC);
    while (ADCSRA & (1 << ADSC));
    return ADC;
}

// Establecer color del led rgb (iluminador)
void rgb_set(uint8_t r, uint8_t g, uint8_t b) {
    PORTB = (PORTB & ~(1 << RED) | (1 << GREEN) | (1 << BLUE)) |
        ((r << RED) | (g << GREEN) | (b << BLUE));
}

// Ciclo de medicion RGB
void rgb_read(void) {

    switch(led_state) {
        case 0:
            rgb_set(1,0,0);
            adc_sample[0] = adc_read(0); // Rojo
            break;

        case 1:
            rgb_set(0,1,0);
            adc_sample[1] = adc_read(0); // Verde
            break;

        case 2:
            rgb_set(0,0,1);
            adc_sample[2] = adc_read(0); // Azul
            break;
    }
}
```

Listing 2. Lectura de colores

A2c. *Convertidor a string*: —

```
// Convierte un valor entero sin signo en un string
void UTOA(uint16_t value, char *buffer) {
    char temp[6];
    int i = 0, j = 0;

    if (value == 0) {
        buffer[0] = '0';
        buffer[1] = '\0';
        return;
    }

    // Convert digits to temp buffer (reversed)
```

```

while (value > 0 && i < sizeof(temp) - 1) {
    temp[i++] = (value % 10) + '0';
    value /= 10;
}

// Reverse digits into final buffer
while (i > 0) buffer[j++] = temp[--i];
buffer[j] = '\0';
}

```

Listing 3. Convertidor de unsigned integer a string

#### A2d. Manejo de tira led WS2812: —

```

void send_bit(uint8_t bitVal){
    if(bitVal){
        PORTD |= (1<<LED_PIN);
        asm volatile (
            "nop\n\t" "nop\n\t" "nop\n\t" "nop\n\t" "nop\n\t" "nop\n\t"
            \t"
            "nop\n\t" "nop\n\t" "nop\n\t" "nop\n\t" "nop\n\t" "nop\n\t";

        PORTD &= ~(1<<LED_PIN);

        asm volatile (
            "nop\n\t" "nop\n\t" "nop\n\t" "nop\n\t" "nop\n\t" "nop\n\t";

        } else {
        PORTD |= (1<<LED_PIN);
        asm volatile (
            "nop\n\t" "nop\n\t" "nop\n\t" "nop\n\t" "nop\n\t" "nop\n\t";

        PORTD &= ~(1<<LED_PIN);
        asm volatile (
            "nop\n\t" "nop\n\t" "nop\n\t" "nop\n\t" "nop\n\t" "nop\n\t";

        \t"
            "nop\n\t" "nop\n\t" "nop\n\t" "nop\n\t" "nop\n\t" "nop\n\t";

        \t");
        }
    }

// Envia un Byte a la tira de leds
// Utiliza cli y sei para un timing preciso
void send_byte(uint8_t byte) {
    cli();
    for (uint8_t i = 0; i < 8; i++) {
        send_bit(byte & 0x80); // Enviar msb
        primero
        byte <<= 1;
    }
    sei();
}

void ws2812_send_pixel(uint8_t r, uint8_t g,
    uint8_t b) {
    // Cambiar orden dependiendo de formato de
    tira led o matriz
    send_byte(g);
    send_byte(b);
    send_byte(r);
}

// Encender n leds del mismo color
void ws2812_fill(uint8_t r, uint8_t g, uint8_t b,
    uint16_t n) {
    cli();
    for (uint16_t i = 0; i < n; i++) {
        ws2812_send_pixel(r, g, b);
    }
    sei();
    ws2812_show();
}

```

```

// Establecer colores predeterminados
void led_strip_set_color(uint8_t color_id) {
    uint8_t r = 0, g = 0, b = 0;

    switch (color_id) {
        case 1: // Rojo
            r = 255; g = 0; b = 0;
            break;

        case 2: // Amarillo
            r = 255; g = 100; b = 0;
            break;

        case 3: // Verde
            r = 0; g = 255; b = 0;
            break;

        case 4: // Azul claro
            r = 0; g = 255; b = 255;
            break;

        case 5: // Violeta
            r = 100; g = 0; b = 100;

            break;

        case 6: // Morado
            r = 200; g = 0; b = 75;

            break;

        case 7: // Blanco
            r = 255; g = 255; b = 255;
            break;

        default: // Apagar LED
            r = g = b = 0;
            break;
    }

    ws2812_fill(r, g, b, 50);
}

```

Listing 4. Manejo de tira led WS2812

#### A2e. Movimiento de servomotor: —

```

// Establecer angulo en el servomotor
void servo_set_angle(uint8_t angle) {
    uint16_t pulse = 1000 + ((uint32_t)angle *
        4000) / 180;
    OCR1A = pulse;
}

```

Listing 5. Movimiento de servomotor

#### A2f. Reconocimiento de colores: —

```

// Mapeado de colores RGB
typedef struct {
    const char *name;
    uint16_t r, g, b;
} ColorRef;

const ColorRef color_refs[] = {
    {"MORADO", 216, 157, 274},
    {"ROJO", 206, 149, 262},
    {"AMARILLO", 196, 99, 105},
    {"VERDE", 272, 192, 152},
    {"AZUL CLARO", 151, 153, 122},
    {"VIOLETA", 253, 275, 338},
}

```

```

    {"BLANCO",    110, 93, 91},
};

// Identificar color a partir de valores rgb.
const char* identify_color(
    uint16_t r,
    uint16_t g,
    uint16_t b
) {
    uint32_t best_dist = 0xFFFFFFFF;
    const char *best_name = "UNKNOWN";

    for (uint8_t i = 0; i < NUM_COLORS; i++) {
        int32_t dr = (int32_t)r - color_refs[i].r;
        int32_t dg = (int32_t)g - color_refs[i].g;
        int32_t db = (int32_t)b - color_refs[i].b;
        uint32_t dist = dr*dr + dg*dg + db*db;

        if (dist < best_dist) {
            best_dist = dist;
            best_name = color_refs[i].name;
        }
    }
    return best_name;
}

```

Listing 6. Reconocimiento de colores

### A2g. Comunicación por USART: —

```

// Imprimir datos parseados por usart
void print_data(const char *color_name) {
    char buf[10];

    // Find by name
    uint16_t ref_r = 0, ref_g = 0, ref_b = 0;
    for (uint8_t i = 0; i < NUM_COLORS; i++) {
        if (strcmp(color_name, color_refs[i].name)
            == 0) {
            ref_r = color_refs[i].r;
            ref_g = color_refs[i].g;
            ref_b = color_refs[i].b;
            break;
        }
    }

    // Delta
    int16_t dR = (int16_t)adc_sample[0] - ref_r;
    int16_t dG = (int16_t)adc_sample[1] - ref_g;
    int16_t dB = (int16_t)adc_sample[2] - ref_b;

    // Printing
    usart_write_str("Fotocelda: ");
    UTOA(adc_sample[0], buf); usart_write_str(buf);
    ;
    usart_write_str(" ");
    UTOA(adc_sample[1], buf); usart_write_str(buf);
    ;
    usart_write_str(" ");
    UTOA(adc_sample[2], buf); usart_write_str(buf);
    ;

    usart_write_str(" Color detectado: ");
    usart_write_str(color_name);

    usart_write_str(" Valor establecido: [");
    UTOA(ref_r, buf); usart_write_str(buf);
    usart_write_str(",");
    UTOA(ref_g, buf); usart_write_str(buf);
    usart_write_str(",");
    UTOA(ref_b, buf); usart_write_str(buf);
    usart_write_str("]");
}

```

```

    usart_write_str(" Delta: [");
    if (dR >= 0) usart_write_str("+");
    else usart_write_str("-");
    UTOA((dR >= 0) ? dR : -dR, buf);
    usart_write_str(buf);
    usart_write_str(",");

    if (dG >= 0) usart_write_str("+");
    else usart_write_str("-");
    UTOA((dG >= 0) ? dG : -dG, buf);
    usart_write_str(buf);
    usart_write_str(",");

    if (dB >= 0) usart_write_str("+");
    else usart_write_str("-");
    UTOA((dB >= 0) ? dB : -dB, buf);
    usart_write_str(buf);
    usart_write_str("]\r\n");
}

```

Listing 7. Comunicación por USART

### A2h. Bucle principal: —

```

// programa principal
int main(void) {
    ws2812_init();
    usart_init();
    adc_init();
    rgb_init();
    servo_init();
    sei();

    while (1) {
        rgb_read();
        _delay_ms(50);
    }
}

```

Listing 8. Bucle principal

### A3. Piano: —

#### A3a. Librerías utilizadas: —

```

#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <avr/pgmspace.h>

```

Listing 9. Librerías utilizadas

#### A3b. Guardado de notas musicales: —

```

#define G4 392
#define Fb4 370
#define E4 330
#define D4 294
// ...

```

Listing 10. Selección dinámica de prescaler

#### A3c. Selección dinámica de prescaler: —

```

void playFrequencyA(uint16_t freq) {
    if (!freq) return;
}

```



```

DDRD |= (1 << PORTD6);

// Elegir prescaler adecuado para esa
frecuencia

uint8_t presc_bits = 0;
uint16_t ocr = 0;

const uint16_t presc_list[] = {8, 64, 256,
1024};
const uint8_t bits_list[] = {0b010, 0b011, 0
b100, 0b101};

for (uint8_t i = 0; i < 4; i++) {
ocr = (F_CPU / (2UL * presc_list[i] * freq
)) - 1;
if (ocr <= 255) {
presc_bits = bits_list[i];
break;
}
}

TCCR0A = (1 << COM0A0) | (1 << WGM01);
TCCR0B = presc_bits;
OCR0A = (uint8_t)ocr;
}

// Dejar de reproducir frecuencia buzzer 1
void stopFrequencyA(void) {
TCCR0A = 0;
TCCR0B = 0;
DDRD |= (1 << PORTD6);
PORTD &= ~(1 << PORTD6);
}

```

Listing 11. Selección dinámica de prescaler

### A3d. Reproducción de pistas: —

```

const int midiA[349][3] PROGMEM = {
{G4, 252, 0},
{Fb4, 252, 0},
{E4, 252, 0},
{E4, 252, 0},
// ...
}

void read_midi_event(const int (*track)[3],
uint16_t index, int out_values[3]) {
for (uint8_t i = 0; i < 3; i++) {
out_values[i] = pgm_read_word(&(track[
index][i]));
}
}

// Reproducir melodía o track en buzzer 1
void playTrackA(void) {
int values[3];
if (song == 0) {
read_midi_event(midiC, indexA++, values);
} else {
read_midi_event(midiA, indexA++, values);
}

uint16_t freq = values[0];
maxCountAon = values[1];
maxCountAoff = values[2];

playFrequencyA(freq);
enableCountAon = 1;
}

```

```

ISR(TIMER1_OVF_vect){
TCNT1 = 65536 - 250; // 1ms preload

// Debouncing
if (debounce_active) { // 1 ms debounce
PCICR |= (1 << PCIE1) | (1 << PCIE2);
debounce_active = 0;
}

// Note playing
if (enableCountAon) {
if (++countA > maxCountAon) {
eventAon = 1;
}
} else if (enableCountAoff){
if (++countA > maxCountAoff){
eventAoff = 1;
}
}

if (enableCountBon) {
if (++countB > maxCountBon) {
eventBon = 1;
}
} else if (enableCountBoff){
if (++countB > maxCountBoff){
eventBoff = 1;
}
}
}
}

```

Listing 12. Reproducción de pistas

### A3e. Manejo de estados por USART: —

```

// Manejo de cambio de estados de usart
void handleUSART(uint8_t character){
if (character == '1'){
mode = 1;
eventAoff = 1;
eventBoff = 0;

song = 0;

eventAon = 0; // Encender track A
indexA = 0; // Posicion track A
countA = 0; // Conteo de overflow de notas
de track A
maxCountAon = 0; // Maximo conteo de
overflow encendido en A
maxCountAoff = 0; // Maximo conteo de
overflow apagado en A
enableCountAon = 0; // Habilitar conteo de
encendido en A
enableCountAoff = 0; // Habilidad conteo
de apagado en A

eventBon = 0; // Encender track B
indexB = 0; // Posicion track B
countB = 0; // Conteo de overflow de notas
de track B
maxCountBon = 0; // Maximo conteo de
overflow encendido en B
maxCountBoff = 0; // Maximo conteo de
overflow apagado en B
enableCountBon = 0; // Habilitar conteo de
encendido en B
enableCountBoff = 0; // Habilidad conteo
de apagado en B
}
}

```

```

PCICR &= ~(1 << PCIE1) | (1 << PCIE2));
stopFrequencyB();

} else if (character == '2'){
    mode = 1;
    eventAoff = 1;
    eventBoff = 1;

    song = 1;

    eventAon = 0; // Encender track A
    indexA = 0; // Posicion track A
    countA = 0; // Conteo de overflow de notas
de track A
    maxCountAon = 0; // Maximo conteo de
overflow encendido en A
    maxCountAoff = 0; // Maximo conteo de
overflow apagado en A
    enableCountAon = 0; // Habilitar conteo de
encendido en A
    enableCountAoff = 0; // Habilidad conteo
de apagado en A

    eventBon = 0; // Encender track B
    indexB = 0; // Posicion track B
    countB = 0; // Conteo de overflow de notas
de track B
    maxCountBon = 0; // Maximo conteo de
overflow encendido en B
    maxCountBoff = 0; // Maximo conteo de
overflow apagado en B
    enableCountBon = 0; // Habilitar conteo de
encendido en B
    enableCountBoff = 0; // Habilidad conteo
de apagado en B

    PCICR &= ~(1 << PCIE1) | (1 << PCIE2));
stopFrequencyA();

} else if (character == 'P'){
    mode = 0;
    eventAoff = 0;
    eventBoff = 0;

    eventAon = 0; // Encender track A
    indexA = 0; // Posicion track A
    countA = 0; // Conteo de overflow de notas
de track A
    maxCountAon = 0; // Maximo conteo de
overflow encendido en A
    maxCountAoff = 0; // Maximo conteo de
overflow apagado en A
    enableCountAon = 0; // Habilitar conteo de
encendido en A
    enableCountAoff = 0; // Habilidad conteo
de apagado en A

    eventBon = 0; // Encender track B
    indexB = 0; // Posicion track B
    countB = 0; // Conteo de overflow de notas
de track B
    maxCountBon = 0; // Maximo conteo de
overflow encendido en B
    maxCountBoff = 0; // Maximo conteo de
overflow apagado en B
    enableCountBon = 0; // Habilitar conteo de
encendido en B
    enableCountBoff = 0; // Habilidad conteo
de apagado en B

    startDebounceTimer();
    stopFrequencyA();
    stopFrequencyB();
}
}

```

Listing 13. Manejo de estados por USART

#### A3f. Programa principal: —

```

// Programa principal
int main(void) {
    timer1_init();
    usart_init_9600();
    init_piano_buttons();
    sei();

    usart_write_str("Elija una opcion:\r\n");
    usart_write_str("[1] Dragon Ball - Cha-La Head
-Cha-La\r\n");
    usart_write_str("[2] Portal - Still alive\r\n"
);
    usart_write_str("[P] Modo piano\r\n");

    while (1){
        if (mode == 0){
            piano_mode();
        } else if (mode == 1){
            song_mode();
        }
        uint8_t c;
        if (usart_read_try(&c)) {
            handleUSART(c);
        }
    }
}

```

Listing 14. Programa principal

#### A4. Cerradura: —

##### A4a. Librerias para lcd: —

```

#include "i2c_master.h"
#include "i2c_master.c"
#include "liquid_crystal_i2c.h"
#include "liquid_crystal_i2c.c"

int main(void){
    LiquidCrystalDevice_t device = lq_init(0x27,
16, 2, LCD_5x8DOTS);
    lq_turnOnBacklight(&device);
    char welcomeText[] = "Bienvenido!";
    lq_print(&device, welcomeText);
    while (1);
}

```

Listing 15. Librerias utilizadas para pantalla LCD

##### A4b. Logica de estados: —

```

typedef enum { UI_MENU, UI_INGRESO,
    UI_CAMBIO_ACTUAL, UI_CAMBIO_NUEVA, UI_ABIERTO,
    UI_ALARMA } ui_state_t;

int main(void){
    while(1){
        char key = keypad_scan();
        if (key) {
            switch (ui_state)
            {
                case UI_MENU: break;
                case UI_INGRESO: break;
                case UI_CAMBIO_ACTUAL: break;
            }
        }
    }
}

```

```

        case UI_CAMBIO_NUEVA: break;
        case UI_CAMBIO_ALARMA: break;
        case UI_CAMBIO_ABIERTO: break;
    }
}

```

Listing 16. Logica de estados

#### A4c. Lectura multiplexada de keypad: —

```

// Mapeo de keypad
const char keypad[4][4] = {
    {'1', '2', '3', 'A'},
    {'4', '5', '6', 'B'},
    {'7', '8', '9', 'C'},
    {'*', '0', '#', 'D'}
};

char keypad_scan(void) {
    if (!keypad_enable) return 0;

    uint8_t row, col;
    uint8_t cols;
    static uint8_t prevKey; // store for later

    for (row = 0; row < 4; row++) {
        PORTD = (PORTD | 0xF0) & ~(1 << (row + 4))
        ;
        _delay_us(5);
        cols = PIND & 0x0F;

        for (col = 0; col < 4; col++) {
            if (!(cols & (1 << col)) ) {
                if ((prevKey == keypad[row][col]))
                    return 0;

                keypad_debounce_ms(200);
                prevKey = keypad[row][col];
                return keypad[row][col];
            }
        }
        prevKey = 0;
        return 0;
    }
}

```

Listing 17. Funcion de lectura multiplexada de keypad

#### A4d. Implementacion de tareas: —

```

uint32_t millis_counter = 0;

uint32_t keypad_on_at = 0;
uint8_t keypad_enable = 1;

ISR(TIMER0_OVF_vect){
    millis_counter++;
}

uint32_t millis_now(void) {
    uint32_t m;
    cli(); // disable interrupts
    m = millis_counter;
    sei(); // re-enable
    return m;
}

void keypad_debounce_ms(uint16_t delay_ms){
    keypad_enable = 0;
    keypad_on_at = millis_now() + delay_ms;
}

```

```

void keypad_task(void){
    if (!keypad_enable && (millis_now() >
        keypad_on_at)){
        keypad_enable = 1;
    }
}

int main(void){
    // main code ...
    while (1){
        keypad_task();
        // loop code ...
    }
}

```

Listing 18. Implementacion de tareas

#### A4e. Escritura y lectura de EEPROM: —

```

#include <avr/eeprom.h>

#define MAX_PASSWORD_LENGTH 6
#define EEPROM_MAGIC 0x42

uint8_t EEMEM ee_magic;
char EEMEM ee_password[MAX_PASSWORD_LENGTH + 1];

char storedPassword[MAX_PASSWORD_LENGTH + 1] = "
123456";
char typedPassword[MAX_PASSWORD_LENGTH + 1];

void eeprom_load_password(void) {
    if (eeprom_read_byte(&ee_magic) !=
        EEPROM_MAGIC) {

        eeprom_update_block(
            storedPassword,
            ee_password,
            sizeof(storedPassword)
        );

        eeprom_update_byte(&ee_magic, EEPROM_MAGIC
        );
    } else {
        eeprom_read_block(
            storedPassword,
            ee_password,
            sizeof(storedPassword)
        );
    }
}

void eeprom_save_password(const char *pwd) {
    char buf[MAX_PASSWORD_LENGTH + 1];
    strncpy(buf, pwd, MAX_PASSWORD_LENGTH);
    buf[MAX_PASSWORD_LENGTH] = '\0';
    eeprom_update_block(buf, ee_password, sizeof(
        buf));
}

```

Listing 19. Escritura y lectura de EEPROM

#### A4f. Tarea de alarma: —

```

void alarm_task(void){
    if (!alarm_active) return;
    uint32_t now = millis_now();

    if (now > alarm_until){
        alarm_active = 0;
        PORTB &= ~(1<<PORTB5);
    }
}

```

```
        led_red_on();  
        return;  
    }  
  
    if (now > alarm_next_toggle){  
        alarm_next_toggle = now + ALARM_TOGGLE_MS;  
        if (alarm_phase){  
            led_red_on();  
        } else {  
            led_green_on();  
        }  
        alarm_phase ^= 1;  
        buzzer_beep(100);  
    }  
}
```

Listing 20. Tarea de alarma